

Project Report

On

Electricity Bill Optimizer using C++ Programming Language

Course No: CSE 2100

Course Name: Object-Oriented Programming Laboratory

Submitted by:

Kankon Mondal

Roll: 2409021

October, 2026

Department of Electronics and Communication Engineering

Khulna University of Engineering & Technology

Khulna-9203, Bangladesh.

Problem Statement

Develop a C++ project to solve a real-life problem using object-oriented programming concepts. The project should provide a practical solution to an existing problem while demonstrating the application of fundamental C++ programming skills.

Open-ended features:

1. Use classes, objects, constructors, destructors, and member functions.
2. Implement inheritance, encapsulation, and polymorphism where appropriate.
3. Use suitable data types, strings, arrays or containers, conditional statements, and loops.
4. Provide a user-friendly, menu-driven command-line interface.
5. Include input validation and appropriate memory management.

Result analysis:

1. Analyze the time required to complete the project.
2. Identify invalid inputs, runtime errors, or unexpected behavior encountered during testing.

Evaluation

Open-ended features	**Remarks**
Classes and Objects	
Member functions	
Constructors and Destructors	
Inheritance and Polymorphism	

	Excellent	Above Average	Average	Below Average	Marks (10)
Organization/Presentation					
Necessary data presentation					
Result Analysis					
Conclusion					

Signature of Examiner

Contents

Problem Statement 2

Evaluation 2

1. Introduction 4

1.1 Motivation and Background 4

1.2 Objectives 4

2. Current Works 5

3. Methodology 5

3.1 Project Diagram 6

3.2 Hardware and Software Requirements 7

4. Implementation and Testing 7

4.1 Full Source-Code 10

4.2 Sample Input and Output 17

5. Result Analysis 18

5.1 Error Handling 19

5.2 Development Time Analysis 19

6. Discussion 20

7. Conclusion 20

7.1 Future Works 20

References 20

1. Introduction

Electricity is an essential part of our daily lives, and managing its consumption efficiently can help reduce household expenses. However, many users are unaware of how much electricity their appliances consume and how their usage affects monthly bills.

The **Electricity Bill Optimizer** is a C++ project developed to address this problem using object-oriented programming. It allows users to add and manage household appliances, calculate estimated monthly electricity bills, analyze energy consumption, and receive recommendations for reducing electricity costs. The system also allows users to modify the electricity rate according to their requirements.

The project demonstrates fundamental OOP concepts, including classes, objects, inheritance, encapsulation, polymorphism, constructors, and destructors. Through a menu-driven command-line interface, it provides a simple and practical approach to understanding and managing household electricity consumption.

1.1 Motivation and Background

The increasing use of electrical appliances in households makes electricity consumption and cost management an important concern. Many users lack a clear understanding of the energy consumed by individual appliances and the impact of their daily usage habits on monthly electricity bills.

The motivation behind this project is to develop a simple software solution that helps users estimate their electricity expenses and identify opportunities to reduce unnecessary consumption. By calculating energy usage and providing appliance-specific recommendations, the system encourages more efficient electricity use.

Additionally, this project provides an opportunity to apply fundamental C++ object-oriented programming concepts to a real-life problem. It connects theoretical programming knowledge with practical implementation through appliance management, bill calculation, and energy consumption analysis.

1.2 Objectives

- To develop a C++ application for managing household electrical appliances.
- To calculate estimated monthly electricity consumption and bills based on appliance usage.
- To analyze energy consumption and identify opportunities for reducing electricity costs.
- To provide appliance-specific recommendations for efficient electricity usage.
- To implement fundamental object-oriented programming concepts including inheritance, encapsulation and polymorphism.
- To develop a menu-driven command-line interface with input validation for user interaction.

2. Current Works

Existing electricity bill management systems often focus primarily on calculating bills from recorded consumption. However, users may also need a simple way to estimate appliance-wise energy consumption and understand how their usage habits affect electricity costs.

The Electricity Bill Optimizer addresses this need by providing a command-line system for adding and viewing appliances, calculating estimated monthly bills, analyzing energy consumption, and generating cost-saving recommendations. It also allows users to adjust the electricity rate for different calculation scenarios.

The current implementation focuses on household electricity management through a simple, interactive interface developed using C++ and object-oriented programming concepts.

3. Methodology

The Electricity Bill Optimizer was developed using C++ by applying object-oriented programming principles to address the practical problem of household electricity consumption and cost management. The system follows a menu-driven approach through which users can manage appliances and analyze their estimated electricity expenses.

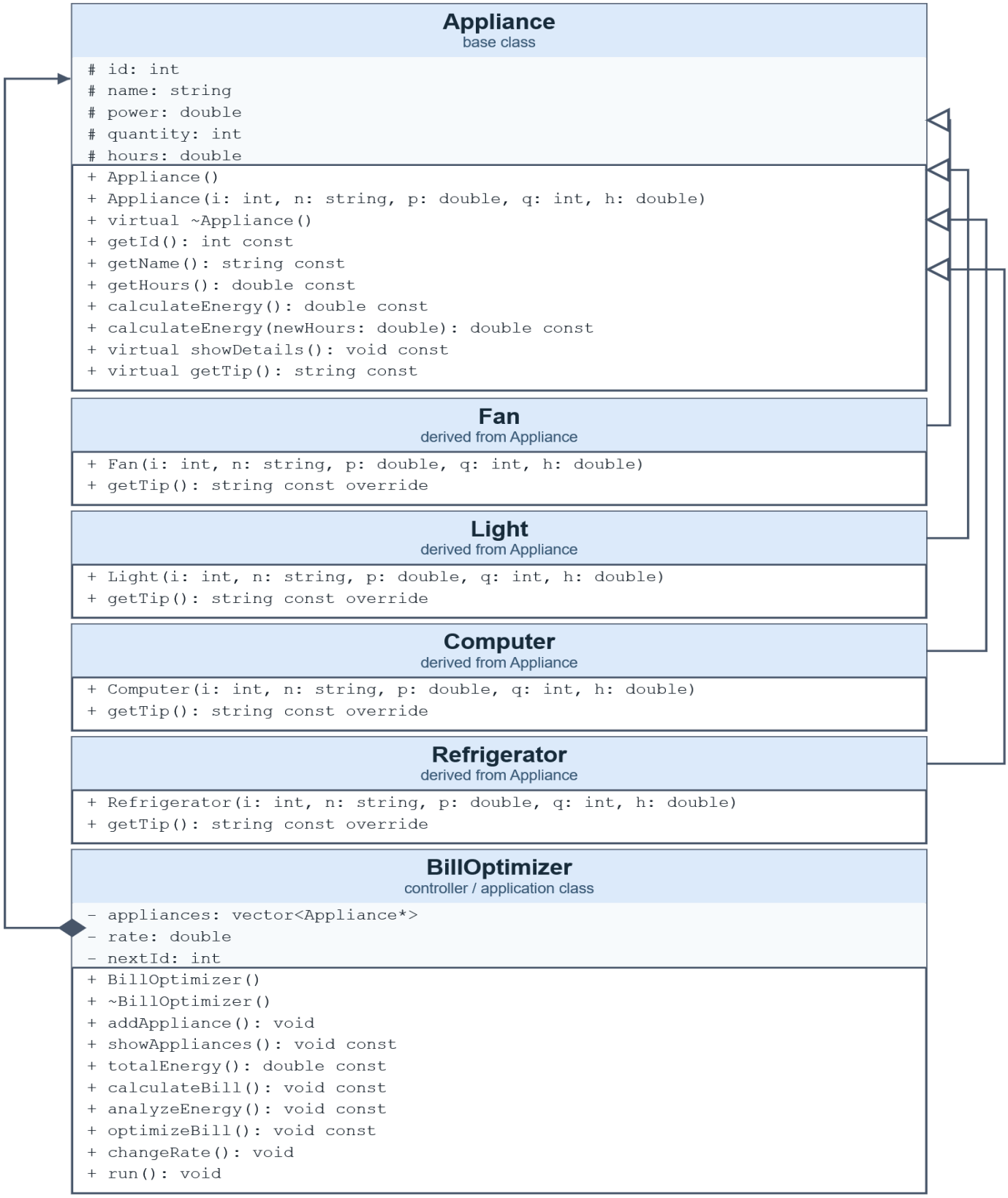
First, an Appliance base class was created to store common information, including appliance ID, name, power rating, quantity, and daily operating hours. Derived classes such as Fan, Light, Computer, and Refrigerator inherit these properties and provide appliance-specific recommendations through polymorphism. Constructors initialize appliance objects, while destructors help manage dynamically allocated memory.

The BillOptimizer class manages the collection of appliances and performs the main calculations. Energy consumption is calculated from the power rating, quantity, and daily operating hours, with the result expressed in kilowatt-hours (kWh). Monthly electricity expenses are then estimated using the configured electricity rate.

The system also analyzes appliance-wise energy consumption and provides recommendations for reducing electricity costs. Users can add appliances, view their details, calculate bills, analyze consumption, optimize electricity usage, and change the electricity rate through the menu.

Finally, the program is tested with different appliance configurations and input values to check the accuracy of calculations, the proper functioning of its features, and its response to invalid inputs. This methodology combines practical problem-solving with the application of fundamental C++ programming and object-oriented design concepts.

3.1 Project Diagram



3.2 Hardware and Software Requirements

Hardware Requirements:

- Computer or Laptop
- Minimum 4 GB RAM
- 128 GB Storage

Software Requirements:

- C Compiler (GCC / CodeBlocks / Dev-C++)
- Operating System (Windows/Linux/MacOS)

4. Implementation and Testing

I. Header Files and Namespace

The program uses standard C++ header files to support input/output, string manipulation, dynamic collections, and formatted output.

```
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>

using namespace std;
```

Explanation: The iostream header handles input and output, string manages text, vector stores multiple appliances, and iomanip formats numerical output. The std namespace allows standard library elements to be used without the std:: prefix.

II. Appliance Class and Data Members

The Appliance class serves as the base class for different household appliances.

```
class Appliance {
protected:
    int id;
    string name;
    double power;
    int quantity;
    double hours;
};
```

Explanation: These data members store the appliance ID, name, power rating in watts, quantity, and daily operating hours. The protected access specifier allows derived classes to access these members.

III. Constructors and Initialization

The default and parameterized constructors initialize appliance objects.

```
Appliance() {
    id = 0;
    name = "Unknown";
    power = 0;
    quantity = 1;
    hours = 0;
}
Light(int i, string n, double p, int q, double h) {
    id = i;
    name = n;
    power = p;
    quantity = q;
    hours = h;
}
```

Explanation: The default constructor assigns initial values, while the parameterized constructor of Light initializes its inherited data members using the supplied arguments.

IV. Member Functions and Energy Calculation

The program calculates monthly energy consumption using the appliance's power rating, quantity, and operating hours.

```
double calculateEnergy() const {
    return (power / 1000.0) * quantity
           * hours * 30;
}
```

Explanation: Power is converted from watts to kilowatts before multiplying by the quantity, daily operating hours, and 30 days. The result represents estimated monthly energy consumption in kWh.

V. Inheritance and Polymorphism

Different appliance classes inherit common properties from the Appliance base class and provide their own recommendations.

```
class Light : public Appliance {
public:
    string getTip() const override {
        if (power > 15)
            return "Consider using an energy-efficient LED
light.";

        return "Switch off unused lights.";
    }
};
```

Explanation: The Light class inherits from Appliance and overrides the virtual getTip() function. This demonstrates inheritance and runtime polymorphism.

VI. Vector and Dynamic Memory Allocation

A vector stores pointers to the appliance objects added by the user.

```
vector<Appliance*> appliances;

appliances.push_back(
    new Fan(nextId, name, power, quantity, hours)
);
```

Explanation: The new operator creates a Fan object dynamically, and push_back() stores its pointer in the vector. This allows the system to manage a changing number of appliances.

VII. Destructor and Memory Management

The BillOptimizer destructor releases the memory allocated for appliance objects.

```
~BillOptimizer() {
    for (int i = 0; i < appliances.size(); i++) {
        delete appliances[i];
    }
}
```

Explanation: The loop visits every appliance pointer and deletes the corresponding object. This prevents memory leaks when the BillOptimizer object is destroyed.

VIII. Monthly Bill Calculation

The program estimates electricity expenses using total energy consumption and the configured electricity rate.

```
double bill = totalEnergy() * rate;

cout << "Estimated bill: Tk "
     << fixed << setprecision(2)
     << bill << endl;
```

Explanation: The total monthly energy consumption is multiplied by the electricity rate per kWh. The fixed and setprecision(2) manipulators display the result with two decimal places.

IX. Conditional Statements and Loops

The program uses conditional statements and loops to control menu navigation and validate user input.

```
switch (choice) {
    case 1:
```

```
            addAppliance();
            break;

        case 2:
            showAppliances();
            break;

        case 3:
            calculateBill();
            break;

        default:
            cout << "Invalid choice." << endl;
    }
```

Explanation: The switch statement executes the appropriate function according to the user's selection. A loop repeatedly displays the menu until the user chooses to exit.

4.1 Full Source-Code

```
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
using namespace std;

const int DAYS_PER_MONTH = 30;

class Appliance {
protected:
    int id;
    string name;
    double power;
    int quantity;
    double hours;

public:
    Appliance() {
        id = 0;
        name = "Unknown";
        power = 0;
        quantity = 1;
        hours = 0;
    }

    Appliance(int i, string n, double p, int q, double h) {
        id = i;
        name = n;
```

```
        power = p;
        quantity = q;
        hours = h;
    }

    virtual ~Appliance() {}

    int getId() const { return id; }
    string getName() const { return name; }
    double getHours() const { return hours; }

    double calculateEnergy() const {
        return (power / 1000.0) * quantity * hours * DAYS_PER_MONTH;
    }

    double calculateEnergy(double newHours) const {
        return (power / 1000.0) * quantity * newHours * DAYS_PER_MONTH;
    }

    virtual void showDetails() const {
        cout << left << setw(5) << id
             << setw(22) << name
             << setw(14) << power
             << setw(8) << quantity
             << setw(10) << hours
             << setw(12) << calculateEnergy() << endl;
    }

    virtual string getTip() const {
        return "Switch off the appliance when it is not needed.";
    }
};

class Fan : public Appliance {
public:
    Fan(int i, string n, double p, int q, double h) {
        id = i;
        name = n;
        power = p;
        quantity = q;
        hours = h;
    }

    string getTip() const override {
        return "Reduce usage when possible and switch it off when not needed.";
    }
};

class Light : public Appliance {
public:
    Light(int i, string n, double p, int q, double h) {
        id = i;
        name = n;
```

```
        power = p;
        quantity = q;
        hours = h;
    }

    string getTip() const override {
        if (power > 15)
            return "Consider using an energy-efficient LED light.";
        return "Use daylight when possible and switch off unused lights.";
    }
};

class Computer : public Appliance {
public:
    Computer(int i, string n, double p, int q, double h) {
        id = i;
        name = n;
        power = p;
        quantity = q;
        hours = h;
    }

    string getTip() const override {
        return "Enable power-saving mode and shut down after use.";
    }
};

class Refrigerator : public Appliance {
public:
    Refrigerator(int i, string n, double p, int q, double h) {
        id = i;
        name = n;
        power = p;
        quantity = q;
        hours = h;
    }

    string getTip() const override {
        return "Avoid frequent door opening and keep space around the
refrigerator.";
    }
};

class BillOptimizer {
private:
    vector<Appliance*> appliances;
    double rate;
    int nextId;

public:
    BillOptimizer() {
        rate = 8.0;
        nextId = 1;
```

```
    }

    ~BillOptimizer() {
        for (int i = 0; i < appliances.size(); i++) {
            delete appliances[i];
        }
    }

    void addAppliance() {
        int type, quantity;
        double power, hours;
        string name;

        cout << "\n1. Fan\n2. Light\n3. Computer\n4. Refrigerator\n";
        cout << "Choose appliance type: ";
        cin >> type;

        if (type < 1 || type > 4) {
            cout << "Invalid appliance type.\n";
            return;
        }

        cout << "Enter appliance name: ";
        cin >> ws;
        getline(cin, name);

        cout << "Enter power in watts: ";
        cin >> power;
        cout << "Enter quantity: ";
        cin >> quantity;
        cout << "Enter daily usage hours (0-24): ";
        cin >> hours;

        if (power <= 0 || quantity <= 0 || hours < 0 || hours > 24) {
            cout << "Invalid input. Appliance was not added.\n";
            return;
        }

        if (type == 1)
            appliances.push_back(new Fan(nextId, name, power, quantity, hours));
        else if (type == 2)
            appliances.push_back(new Light(nextId, name, power, quantity,
hours));
        else if (type == 3)
            appliances.push_back(new Computer(nextId, name, power, quantity,
hours));
        else
            appliances.push_back(new Refrigerator(nextId, name, power, quantity,
hours));

        cout << "Appliance added. ID: " << nextId << endl;
        nextId++;
    }
```

```
void showAppliances() const {
    if (appliances.empty()) {
        cout << "\nNo appliances added yet.\n";
        return;
    }

    cout << "\n" << left << setw(5) << "ID"
         << setw(22) << "Name"
         << setw(14) << "Power(W)"
         << setw(8) << "Qty"
         << setw(10) << "Hrs/day"
         << setw(12) << "kWh/month" << endl;

    cout << string(71, '-') << endl;

    for (int i = 0; i < appliances.size(); i++) {
        appliances[i]->showDetails();
    }
}

double totalEnergy() const {
    double total = 0;
    for (int i = 0; i < appliances.size(); i++) {
        total += appliances[i]->calculateEnergy();
    }
    return total;
}

void calculateBill() const {
    double energy = totalEnergy();
    cout << "\n------ MONTHLY BILL ESTIMATE ------\n";
    cout << "Monthly energy: " << fixed << setprecision(2)
         << energy << " kWh\n";
    cout << "Rate: Tk " << rate << " per kWh\n";
    cout << "Estimated bill: Tk " << energy * rate << endl;
    cout << "Note: This estimate excludes taxes and extra charges.\n";
}

void analyzeEnergy() const {
    if (appliances.empty()) {
        cout << "\nPlease add appliances first.\n";
        return;
    }

    int highestIndex = 0;
    cout << "\n------ ENERGY ANALYSIS ------\n";

    for (int i = 0; i < appliances.size(); i++) {
        cout << appliances[i]->getName() << ": "
             << fixed << setprecision(2)
             << appliances[i]->calculateEnergy() << " kWh/month\n";
```

```
        if (appliances[i]->calculateEnergy() >
            appliances[highestIndex]->calculateEnergy()) {
            highestIndex = i;
        }
    }

    cout << "\nHighest energy consumer: "
         << appliances[highestIndex]->getName() << endl;
    cout << "Suggestion: "
         << appliances[highestIndex]->getTip() << endl;
}

void optimizeBill() const {
    if (appliances.empty()) {
        cout << "\nPlease add appliances first.\n";
        return;
    }

    double oldTotal = 0;
    double newTotal = 0;

    cout << "\n------ SAVING SUGGESTIONS ------\n";
    for (int i = 0; i < appliances.size(); i++) {
        double oldEnergy = appliances[i]->calculateEnergy();
        double newHours = appliances[i]->getHours();

        if (newHours > 1)
            newHours = newHours - 1;

        double newEnergy = appliances[i]->calculateEnergy(newHours);
        oldTotal += oldEnergy;
        newTotal += newEnergy;

        cout << "\n" << appliances[i]->getName() << ":\n";
        cout << "Current use: " << fixed << setprecision(2)
             << oldEnergy << " kWh/month\n";

        if (newHours < appliances[i]->getHours()) {
            cout << "Suggested use: " << newHours << " hours/day\n";
            cout << "Possible saving: " << oldEnergy - newEnergy
                 << " kWh/month\n";
            cout << "Possible bill saving: Tk "
                 << (oldEnergy - newEnergy) * rate << endl;
        } else {
            cout << "Usage is already 1 hour/day or less.\n";
        }
    }

    cout << "\n------ SUMMARY ------\n";
    cout << "Current energy: " << fixed << setprecision(2)
         << oldTotal << " kWh/month\n";
    cout << "After changes: " << newTotal << " kWh/month\n";
    cout << "Possible monthly saving: Tk "
```

```
             << (oldTotal - newTotal) * rate << endl;
        cout << "This is an estimate, not a guaranteed saving.\n";
    }

    void changeRate() {
        double newRate;
        cout << "Enter electricity rate per kWh (Tk): ";
        cin >> newRate;

        if (newRate < 0) {
            cout << "Rate cannot be negative.\n";
            return;
        }

        rate = newRate;
        cout << "Electricity rate updated.\n";
    }

    void run() {
        int choice;
        do {
            cout << "\n====================================\n";
            cout << "      ELECTRICITY BILL OPTIMIZER\n";
            cout << "====================================\n";
            cout << "1. Add an appliance\n";
            cout << "2. View all appliances\n";
            cout << "3. Calculate monthly bill\n";
            cout << "4. Analyze energy consumption\n";
            cout << "5. Optimize electricity cost\n";
            cout << "6. Change electricity rate\n";
            cout << "7. Exit\n";
            cout << "Enter your choice: ";
            cin >> choice;

            switch (choice) {
                case 1: addAppliance(); break;
                case 2: showAppliances(); break;
                case 3: calculateBill(); break;
                case 4: analyzeEnergy(); break;
                case 5: optimizeBill(); break;
                case 6: changeRate(); break;
                case 7: cout << "Thank you for using the program!\n"; break;
                default: cout << "Invalid choice. Try again.\n";
            }
        } while (choice != 7);
    }
};

int main() {
    BillOptimizer app;
    app.run();
    return 0;
}
```

4.2 Sample Input Output

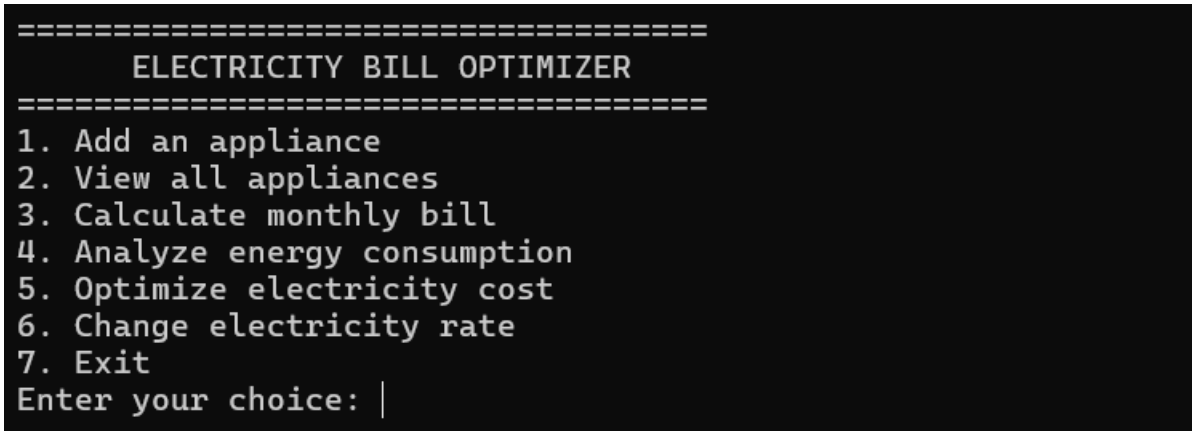


Fig-02: Basic Interface

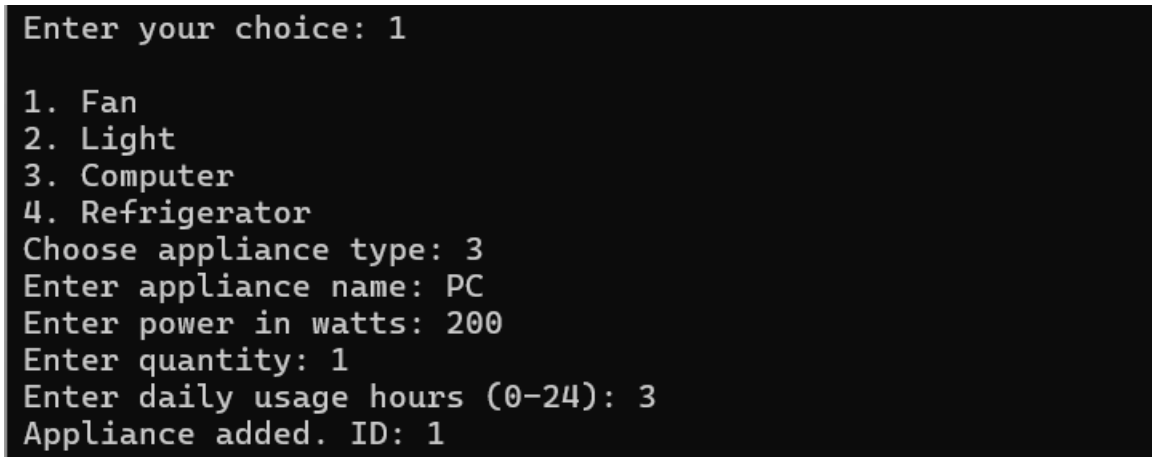


Fig-03: Adding an Appliance

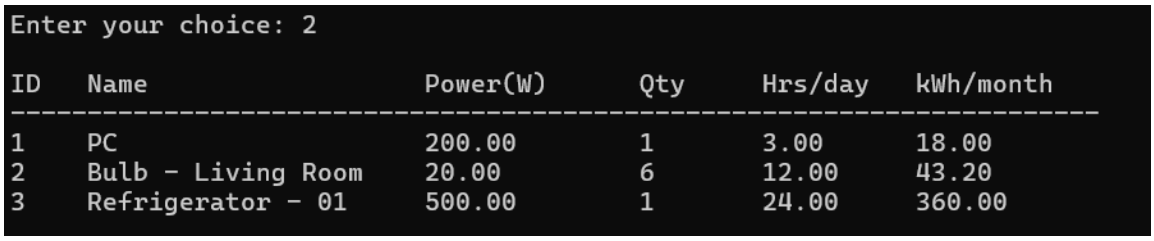


Fig-04: Viewing all Appliances

```
Enter your choice: 3

------ MONTHLY BILL ESTIMATE ------
Monthly energy: 421.20 kWh
Rate: Tk 8.00 per kWh
Estimated bill: Tk 3369.60
Note: This estimate excludes taxes and extra charges.
```

Fig-05: Monthly Bill Calculation

```
Enter your choice: 5

------ SAVING SUGGESTIONS ------

PC:
Current use: 18.00 kWh/month
Suggested use: 2.00 hours/day
Possible saving: 6.00 kWh/month
Possible bill saving: Tk 48.00

Bulb - Living Room:
Current use: 43.20 kWh/month
Suggested use: 11.00 hours/day
Possible saving: 3.60 kWh/month
Possible bill saving: Tk 28.80

Refrigerator - 01:
Current use: 360.00 kWh/month
Suggested use: 23.00 hours/day
Possible saving: 15.00 kWh/month
Possible bill saving: Tk 120.00

------ SUMMARY ------
Current energy: 421.20 kWh/month
After changes: 396.60 kWh/month
Possible monthly saving: Tk 196.80
This is an estimate, not a guaranteed saving.
```

Fig-06: Optimizing Electricity Cost

5. Result Analysis

The Electricity Bill Optimizer was successfully developed using C++ and object-oriented programming principles. The system provides a menu-driven interface for adding household appliances, viewing appliance details, calculating estimated monthly electricity bills, analyzing energy consumption, and generating cost-saving recommendations.

During testing, the program was evaluated with different appliance configurations, power ratings, quantities, and daily operating hours. The calculations demonstrated how appliance power consumption and usage duration affect monthly electricity expenses. The electricity rate can also be changed, allowing users to compare estimated costs under different rates.

The program uses inheritance and polymorphism to provide appliance-specific recommendations, while constructors and destructors support object initialization and dynamic memory management. Conditional statements and loops control menu navigation and validate several user inputs.

Some limitations were also identified. The program assumes a 30-day month and constant daily appliance usage. It estimates electricity costs using a single rate and does not account for additional charges or slab-based tariffs. Furthermore, non-numeric input is not fully handled, and appliance data is not permanently stored between program executions.

Overall, the project demonstrates the practical application of C++ OOP concepts to a real-life problem. It provides a useful foundation for understanding household electricity consumption and identifying potential ways to reduce electricity expenses.

.

5.1 Error Handling

During testing, the program successfully handled various scenarios:

- **Invalid Menu Selection:** Displays an error message when the user enters an unsupported menu option.
- **Invalid Appliance Type:** Rejects appliance types that are not supported by the program.
- **Invalid Power Rating:** Prevents zero or negative power ratings.
- **Invalid Quantity:** Rejects zero or negative appliance quantities.
- **Invalid Operating Hours:** Ensures daily operating hours remain within the range of 0 to 24 hours.
- **Electricity Rate Validation:** Prevents negative electricity rates.
- **Input Limitations:** Non-numeric input is not fully handled and may cause unexpected behavior.

5.2 Development Time Analysis

The development of the Electricity Bill Optimizer involved several stages, including planning, designing classes, implementing core functionalities, testing calculations, and debugging errors.

The time required to complete the project was approximately:

- Design and planning: 2–3 hours
- Coding and debugging: 5–6 hours
- Testing and improvement: 2–3 hours

The project required approximately **10 hours** to complete. Most of the time was spent implementing object-oriented features, integrating the bill calculation and optimization functions, and testing the program. The development process also helped improve practical knowledge of C++ programming and problem-solving.

6. Discussion

The Electricity Bill Optimizer demonstrates how C++ and object-oriented programming can be applied to household electricity management. The program successfully integrates appliance management, energy consumption analysis, bill calculation, and cost-saving recommendations into a single command-line application.

The use of inheritance and polymorphism makes it possible to provide recommendations for different types of appliances, while constructors and destructors support object initialization and memory management. Testing also highlights the importance of input validation and accurate calculations in developing reliable software.

Although the system provides useful estimates, its accuracy depends on the information entered by the user. Future improvements could include permanent data storage, more comprehensive input validation, and support for different electricity tariff structures. Overall, the project provides practical experience in applying fundamental C++ concepts to solve a real-life problem.

7. Conclusion

The Electricity Bill Optimizer was successfully developed using C++ and object-oriented programming concepts to manage household appliances, estimate monthly electricity bills, analyze energy consumption, and suggest potential cost-saving measures. Through this project, practical applications of classes, objects, inheritance, polymorphism, constructors, destructors, and dynamic memory management were explored. Although the system has some limitations, such as the lack of permanent data storage and support for complex electricity tariffs, it achieves its primary objectives and demonstrates how programming can help address real-life problems and promote efficient electricity consumption.

7.1 Future Work

- **Data Persistence:** Implement file handling or a database to save appliance information between program executions.
- **Improved Input Validation:** Handle non-numeric inputs and prevent unexpected program behavior.
- **Usage History:** Maintain monthly records to compare electricity consumption over time.
- **Enhanced Optimization:** Provide more detailed recommendations based on appliance usage patterns.
- **Graphical User Interface:** Develop a graphical interface to make the system more user-friendly.

References

[1] Dennis Ritc Bjarne Stroustrup, Programming: Principles and Practice Using C++, 2nd Edition, Addison-Wesley, 2014.

[2] E. Balagurusamy, Object-Oriented Programming with C++, 8th Edition, McGraw Hill Education, 2020.

[3] cppreference.com, “C++ Language Reference.” Available at: https://en.cppreference.com/